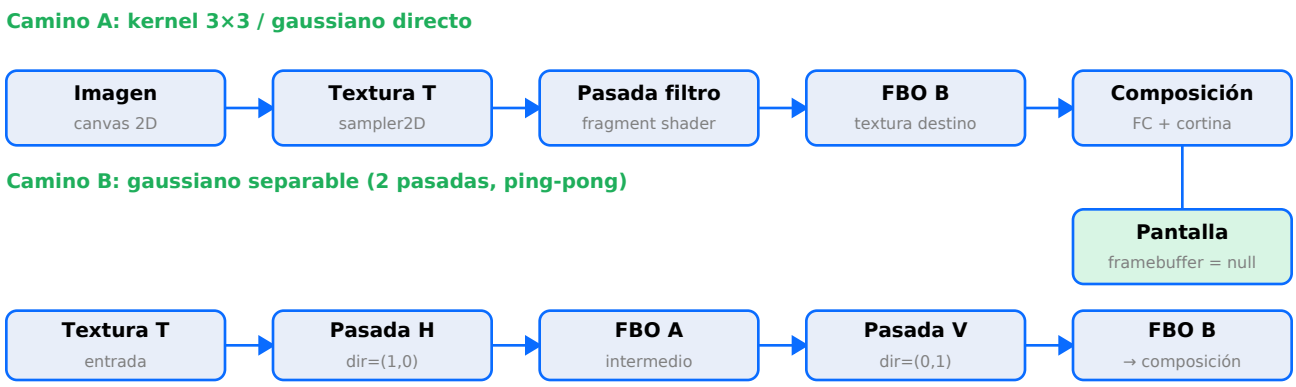
Filtrado Espacial y Convolución 2D en GPU

Cómo funciona la parte gráfica del laboratorio index.html (literate programming)

1. Visión general del pipeline gráfico

Aquí no se dibujan modelos 3D: se dibuja un solo rectángulo que cubre toda la pantalla y el trabajo ocurre en el fragment shader, que la GPU ejecuta una vez por píxel y en paralelo. Cada ejecución lee unos cuantos texels de una textura, los combina con pesos y devuelve un color. Eso es una convolución 2D.

El resultado de un filtro no se muestra directamente: se escribe en una textura intermedia (render-to-texture mediante un FBO) y una última pasada, la composición, decide qué se ve en cada mitad de la cortina. Así el original y el filtrado conviven en pantalla.



2. El quad a pantalla completa y el vertex shader

Cuatro vértices en coordenadas de recorte (−1…+1) forman dos triángulos (TRIANGLE_STRIP). El vertex shader solo reenvía la posición y convierte ese rango a coordenadas de textura uv en [0,1]. Como el quad cubre todo el viewport, cada píxel de salida recibe un uv único e interpolado: es la "dirección" del texel que le corresponde.

```
const VS=`attribute vec2 p;varying vec2 v;void main(){v=p*.5+.5;gl_Position=vec4(p,0.,1.);}`;
gl.bindBuffer(gl.ARRAY_BUFFER,gl.createBuffer());
gl.bufferData(gl.ARRAY_BUFFER,new Float32Array([-1,-1,1,-1,-1,1,1,1]),gl.STATIC_DRAW);
gl.enableVertexAttribArray(0);gl.vertexAttribPointer(0,2,gl.FLOAT,false,0,0);
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL,true);
```

3. Texturas, texels y render-to-texture

La imagen se sube a una textura. Se usa NEAREST para leer el valor exacto de cada texel (sin que la interpolación bilineal mezcle vecinos y falsee la convolución) y CLAMP_TO_EDGE para que, en los bordes, un vecino fuera de la imagen repita el píxel extremo. El volteo en Y al subir la imagen hace que uv.y crezca hacia arriba, como en pantalla.

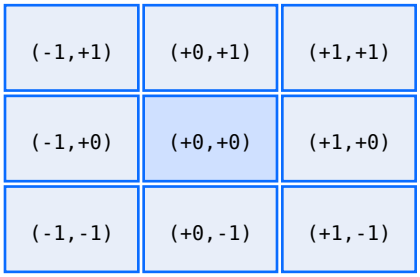
Un FBO es un framebuffer cuyo color es una textura: dibujar en él equivale a calcular una imagen nueva que luego otra pasada puede leer. Es la pieza que permite encadenar pasadas (filtro → composición, o horizontal → vertical).

```
function tex(w,h,s){
 const t=gl.createTexture();gl.bindTexture(gl.TEXTURE_2D,t);
```

```
 [[gl.TEXTURE_MIN_FILTER,gl.NEAREST],[gl.TEXTURE_MAG_FILTER,gl.NEAREST],
  [gl.TEXTURE_WRAP_S,gl.CLAMP_TO_EDGE],[gl.TEXTURE_WRAP_T,gl.CLAMP_TO_EDGE]]
  .forEach(a=>gl.texParameteri(gl.TEXTURE_2D,a[0],a[1]));
 if(s)gl.texImage2D(gl.TEXTURE_2D,0,gl.RGBA,gl.RGBA,gl.UNSIGNED_BYTE,s);
 else gl.texImage2D(gl.TEXTURE_2D,0,gl.RGBA,w,h,0,gl.RGBA,gl.UNSIGNED_BYTE,null);
 return t;}
function fbo(w,h){
 const t=tex(w,h),f=gl.createFramebuffer();
 gl.bindFramebuffer(gl.FRAMEBUFFER,f);
 gl.framebufferTexture2D(gl.FRAMEBUFFER,gl.COLOR_ATTACHMENT0,gl.TEXTURE_2D,t,0);
 return{f,t};}
```

4. Fragment shader del kernel 3×3

Para cada píxel se recorre su vecindad de 3×3. El tamaño de un texel en coordenadas uv es 1/ancho × 1/alto (uniform px), por lo que sumar `(ox,oy)*px` a uv salta exactamente ox píxeles en x y oy en y. Cada color vecino se multiplica por su peso k[i] y se acumula.



Vecino i del bucle: (ox, oy) = (i mod 3 − 1, 1 − i div 3)

`uv_vecino = uv + (ox, oy) · px  con px = (1/W, 1/H)`

Un paso de 1/W en uv equivale exactamente a 1 píxel en x.

El centro (0,0) es el propio píxel; los otros 8 son su vecindad.

```
const FK=HD+`uniform float k[9];uniform float bias;uniform int sob;
float L(vec3 c){return dot(c,vec3(.299,.587,.114));}
void main(){
 vec3 s=vec3(0.);float gx=0.,gy=0.;
 for(int i=0;i<9;i++){
  float ox=float(i-(i/3)*3)-1.,oy=1.-float(i/3);
  vec3 c=texture2D(t,v+vec2(ox,oy)*px).rgb;
  s+=k[i]*c;
  float l=L(c);gx+=l*ox*(2.-abs(oy));gy+=l*oy*(2.-abs(ox));
 }
 gl_FragColor=vec4(sob==1?vec3(sqrt(gx*gx+gy*gy)):s+bias,1.);
}`;
```

5. Sobel: gradiente espacial

En el mismo bucle se acumulan dos derivadas sobre la luminancia (0.299 R + 0.587 G + 0.114 B): Gx detecta cambios horizontales y Gy verticales. Sus pesos se obtienen del propio offset: Gx usa ox·(2−|oy|) y Gy usa oy·(2−|ox|), que reproduce las máscaras clásicas de Sobel. La salida es la magnitud √(Gx²+Gy²): clara donde hay bordes, oscura en zonas lisas.

6. Gaussiano: directo contra separable

El kernel gaussiano 2D es el producto de dos curvas 1D, K(i,j)=k(i)·k(j). Por eso se puede calcular de dos maneras con el mismo resultado: una pasada con doble bucle (N² lecturas) o dos pasadas de bucle simple (2N lecturas): primero horizontal hacia el FBO A y luego vertical hacia el FBO B. El peso se calcula con la fórmula exp(−i²/2σ²)/Z dentro del shader, y `direct` elige el modo. Los límites de los bucles son constantes (−4…4) porque GLSL ES 1.0 lo exige; el radio real R se aplica con `continue`.




```
const FG=HD+`uniform vec2 dir;uniform vec2 g;uniform int R,direct;
float w(int i){return exp(-float(i*i)*g.x)*g.y;}
void main(){vec3 s=vec3(0.);
 if(direct==1){for(int j=-4;j<=4;j++){if(j<-R||j>R)continue;
  for(int i=-4;i<=4;i++){if(i<-R||i>R)continue;
   s+=w(i)*w(j)*texture2D(t,v+vec2(i,j)*px).rgb;}}}
 else{for(int i=-4;i<=4;i++){if(i<-R||i>R)continue;
   s+=w(i)*texture2D(t,v+float(i)*dir*px).rgb;}}
 gl_FragColor=vec4(s,1.);}`;
```

7. Orquestación de las pasadas

La función `draw` prepara una pasada: elige el framebuffer de destino, el programa, la textura de entrada y el tamaño de texel. `render` decide qué pasadas ejecutar según el modo y repite el filtro `rep` veces cuando se sube la "carga" (útil para que la diferencia N^2 contra $2N$ se note en milisegundos). El tiempo se mide con `gl.finish()`, que espera a que la GPU termine; es una medida aproximada pero suficiente para comparar.

```
function draw(P,tx,fb,fn){
 gl.bindFramebuffer(gl.FRAMEBUFFER,fb);gl.viewport(0,0,W,H);gl.useProgram(P.p);
 gl.activeTexture(gl.TEXTURE0);gl.bindTexture(gl.TEXTURE_2D,tx);
 gl.uniform1i(P.u.t,0);gl.uniform2f(P.u.px,1/W,1/H);fn(P.u);
 gl.drawArrays(gl.TRIANGLE_STRIP,0,4);}
let ms=0;
function render(){
 const t0=performance.now(),R=(N-1)/2,sg=Math.max(.7,N/4),i2=1/(2*sg*sg);
 let Z=0;for(let i=-R;i<=R;i++)Z+=Math.exp(-i*i*i2);
 const gset=(u,d,dr)=>{gl.uniform2f(u.dir,d[0],d[1]);gl.uniform2f(u.g,i2,1/Z);
  gl.uniform1i(u.R,R);gl.uniform1i(u.direct,dr);};
 const bias=$('#bias').checked?.5:0;
 for(let r=0;r<rep;r++){
  if(mode==='k')draw(PK,T,B.f,u=>{gl.uniform1fv(u['k[0]'],K);gl.uniform1f(u.bias,bias);gl.uniform1i(u.sob,sob);});
  else if(mode==='d')draw(PG,T,B.f,u=>gset(u,[1,0],1));
  else{draw(PG,T,A.f,u=>gset(u,[1,0],0));draw(PG,A.t,B.f,u=>gset(u,[0,1],0));}
 }
 draw(PC,T,null,u=>{
  gl.activeTexture(gl.TEXTURE1);gl.bindTexture(gl.TEXTURE_2D,B.t);
  gl.uniform1i(u.a,0);gl.uniform1i(u.b,1);
  gl.uniform1f(u.sp,$('#sp').checked?+$('#sx').value:-1);gl.uniform1f(u.lw,1.5/W);
  gl.activeTexture(gl.TEXTURE0);});
 gl.finish();ms=ms*.9+(performance.now()-t0)*.1;}
```

8. Composición: la cortina A/B

La última pasada se dibuja al framebuffer de pantalla. Recibe dos texturas, la original T y la filtrada B, y según la posición horizontal uv.x respecto a la cortina `sp` elige de cuál leer. Una franja estrecha alrededor de `sp` se pinta de cian como guía. Si la cortina está desactivada, `sp` vale -1 y siempre se ve la imagen filtrada.

```
const FC=`precision highp float;varying vec2 v;uniform sampler2D a,b;uniform float sp,lw;
void main(){vec3 c=v.x<sp?texture2D(a,v).rgb:texture2D(b,v).rgb;
 if(abs(v.x-sp)<lw)c=vec3(0.,.9,1.);gl_FragColor=vec4(c,1.);}`;
```

9. Cuantización, recorte y bias

Las texturas y el framebuffer guardan 8 bits por canal en [0,1]. Cada escritura recorta a ese rango, de modo que un filtro con valores negativos (Laplaciano, Sobel con signo, paso alto) pierde su mitad

negativa. Sumar un bias de 0.5 antes de escribir mueve el cero a gris medio: lo oscuro representa negativos y lo claro positivos. De igual forma, si la suma de pesos supera 1 el brillo se satura en blanco, y si es 0 el filtro anula las zonas lisas. La suma de pesos ΣK es la ganancia a frecuencia cero del filtro.